

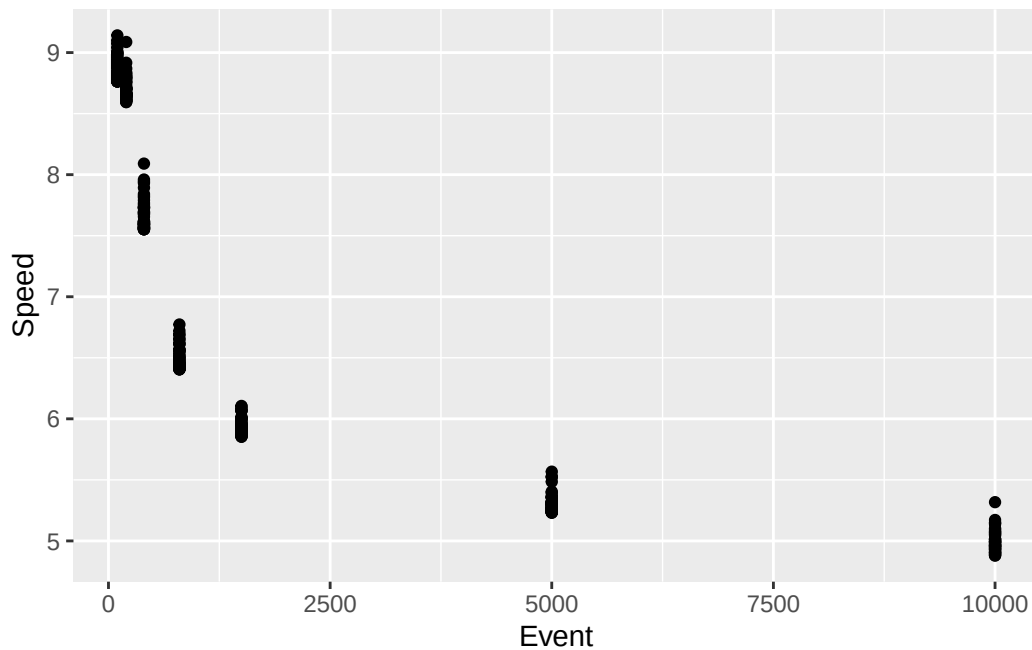
2025 Women's Track Running: Using Non-Linear Least Squares Regression

```
library(tidyverse)
library(ggplot2)

track <- read.csv("WomensTrack2025.csv")
```

1. Make a scatterplot using distance (Event) to predict speed.

```
ggplot(track, aes(x = Event, y = Speed)) +
  geom_point()
```



We will start by applying an logarithmic decay model to the data. A logarithmic decay model has the following form:

$$y = a - b \ln(x)$$

Where a is the value of y when $x = 1$ (where the decay curve starts) and b is how fast the value decreases (and for decay must be positive, as the negative is already included in the equation). As b is larger, the drop will be steeper at first. The $\ln(x)$ term allows the decay to slow over time.

```
logarithmic <- function(x, a, b) { a-b*log(x)}
```

The above code defines the logarithmic decay function, allowing us to use it within the `nls` function, as in the code below. For `nls` models, you must provide starting values for each of the parameters; these values do not have to be exact, they just have to be reasonable guesses. Here, we will guess 10 for a as we know that it must be higher than 9 (as that is the value when $x = 100$), but we cannot see how much higher. For b we will guess 1 as the curve is not too extreme.

```
nls_mod <- nls(Speed ~ logarithmic(Event, a, b),
              data = track,
              start = list(a = 10, b = 1))
```

2. Use the model summary to write out the fitted model.

```
summary(nls_mod)
```

Formula: Speed ~ logarithmic(Event, a, b)

Parameters:

	Estimate	Std. Error	t value	Pr(> t)
a	13.12022	0.08295	158.18	<2e-16 ***
b	0.92106	0.01190	77.42	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3448 on 349 degrees of freedom

Number of iterations to convergence: 1

Achieved convergence tolerance: 1.975e-08

$$\widehat{\text{Speed}} = 13.12 - 0.92 * \ln(\text{Event})$$

3. Report the AIC and BIC for the model.

```
AIC(nls_mod)
```

```
[1] 252.7047
```

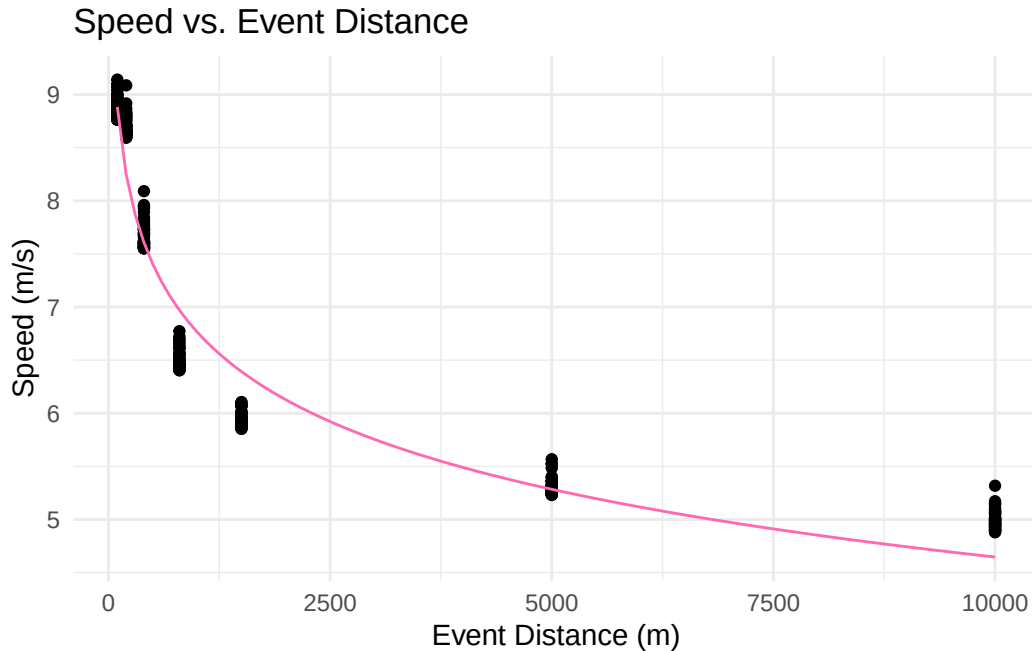
```
BIC(nls_mod)
```

```
[1] 264.2871
```

The AIC for the model is 252.70 and the BIC is 264.29.

4. Plug in the correct values for a and b so the code below will create a scatterplot that shows the logarithmic decay line with the data.

```
a = 13.12
b = 0.92
ggplot(track, aes(x = Event)) +
  geom_point(aes(y = Speed)) +
  geom_function(fun = function(x) { a-b*log(x) }, color="hotpink") +
  labs(title = "Speed vs. Event Distance",
       x = "Event Distance (m)",
       y = "Speed (m/s)") +
  theme_minimal()
```



Now, you will repeat the process to apply a power decay model to the data. A power decay regression equation has the following form:

$$y = ax^{-b}$$

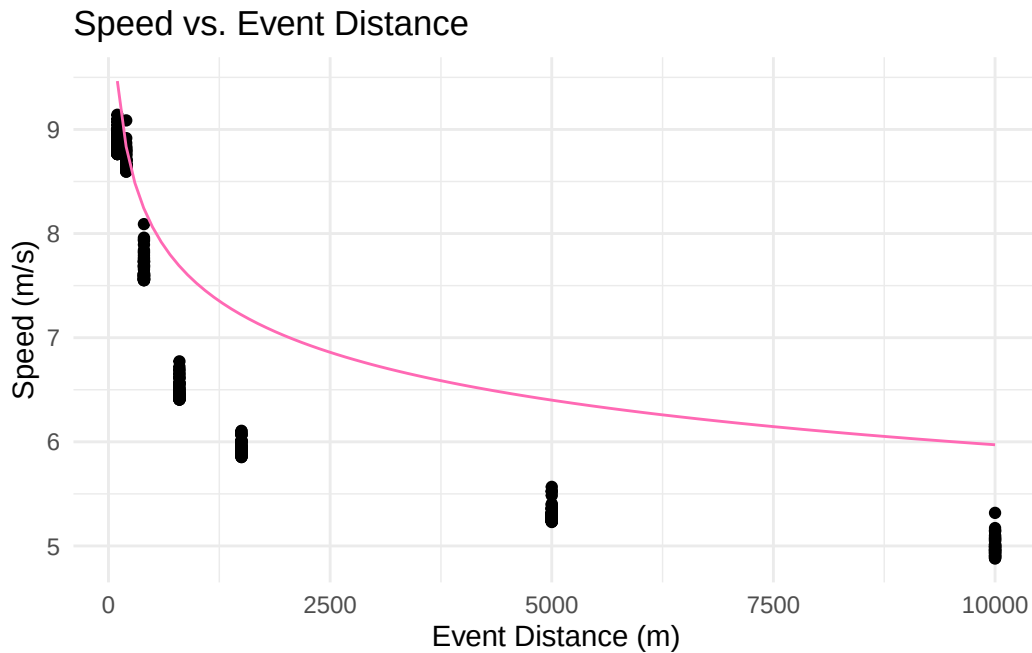
The parameters in the power decay model have similar roles to those in the logarithmic decay model. a impacts the vertical scale of the curve and is typically the y value where the curvature begins. b is the decay rate and changes how quickly the decay happens (and for decay must be positive, as the negative is already included in the equation). Because x is being raised to a negative value, the decay term will eventually converge towards 0.

5. Plug in the correct function and use the scatterplot to test different a and b values to find good starting guesses for when we fit the model.

```
## the answer key uses a = 15 and b = 0.1
## which are reasonable but not exactly correct

a = 15
b = .1
ggplot(track, aes(x = Event)) +
  geom_point(aes(y = Speed)) +
  geom_function(fun = function(x) { a*(x^(-b)) }, color="hotpink") +
  labs(title = "Speed vs. Event Distance",
```

```
x = "Event Distance (m)",
y = "Speed (m/s)" +
theme_minimal()
```



6. Fit the model using the correct function and some reasonable starting values you found.

```
power_curve <- function(x, a, b) { a*(x^(-b)) }

nls_mod2 <- nls(Speed ~ power_curve(Event, a, b),
               data = track,
               start = list(a = 15, b = .1))
```

7. Use the model summary to write out the fitted model.

```
summary(nls_mod2)
```

Formula: Speed ~ power_curve(Event, a, b)

Parameters:

	Estimate	Std. Error	t value	Pr(> t)
a	17.29339	0.16991	101.78	<2e-16 ***

```
b 0.13942    0.00155    89.94    <2e-16 ***
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 0.2871 on 349 degrees of freedom
```

```
Number of iterations to convergence: 4
```

```
Achieved convergence tolerance: 3.833e-08
```

$$\widehat{\text{Speed}} = 17.29 * \text{Event}^{-0.14}$$

8. Report the AIC and BIC for the model.

```
AIC(nls_mod2)
```

```
[1] 123.9476
```

```
BIC(nls_mod2)
```

```
[1] 135.53
```

The AIC for this model is 123.95 and the BIC for this model is 135.53.

9. Which of the two non-linear least squares models you created do you prefer?

The lower AIC and BIC of the second model make it the more useful model for predicting a runner's speed using the distance they are running. I prefer the power decay model.

10. Define a model that uses natural log of **Event** to predict the natural log of **Speed**.

```
transform_mod <- lm(log(Speed) ~ log(Event), track)
summary(transform_mod)
```

Call:

```
lm(formula = log(Speed) ~ log(Event), data = track)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.065206	-0.030988	-0.000751	0.034072	0.099712

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	2.829909	0.009810	288.49	<2e-16 ***
log(Event)	-0.136421	0.001407	-96.97	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.04078 on 349 degrees of freedom

Multiple R-squared: 0.9642, Adjusted R-squared: 0.9641

F-statistic: 9402 on 1 and 349 DF, p-value: < 2.2e-16

10.1. Do you prefer the transformation model or the nls model you picked in question 9 for predicting Speed using Event?

```
AIC(transform_mod)
```

```
[1] -1245.948
```

```
BIC(transform_mod)
```

```
[1] -1234.365
```

The double transformation model has a very low AIC (-1245.9), making it the most useful model, I prefer it for this. However, it is important to remember that using log transformations makes the data harder to work with **in context** and makes visualizations and interpretations less meaningful.